WASP Course Software Engineering assignment

Ioannis Savvas
Department of Computer Science and Engineering, Chalmers University of Technology
savvasi@chalmers.se

September 5, 2026

1 Introduction

My research focuses on developing machine learning techniques for the design of polymer electrolytes. These materials are promising substitutes for small-molecule electrolytes in Li-ion battery cells because of their enhanced safety and sustainability. Yet, their functional properties (e.g ionic conductivity) are still well short of the required application level, making it challenging to identify candidates for real-world use. Machine learning can help accelerate the process of discovery by exploring the chemical space more efficiently before carrying out laboratory synthesis.

In particular, I am using generative models to accelerate solid polymer electrolyte discovery. Polymers consist of repeating units arranged in a sequence, and both the unit and the sequence can be represented as strings or graphs. This makes them well-suited for generative modelling and, in particular, autoregressive models for generating those structures token by token. In parallel, machine learning models are used to predict the properties of interest by learning a mapping between structure and properties. At my current stage, I'm combining autoregressive models and predictive models in a reinforcement learning loop to generate valid polymeric structures with optimized properties that meet the multi-objective criteria for battery applications.

2 Lecture Principles

2.1 Curating validation sets for ML testing

As part of my pipeline, I train models to predict a chemical property based on the polymer structure. A major bottleneck here is data scarcity. The process of accurately generating labeled data is costly (e.g., lab experiments, molecular simulations), and thus we often work with small datasets of ~100–5,000 instances. For that reason, the validation process should be very detailed to ensure that results can be trusted. To give a concrete example, a model that predicts a polymer property on the validation set fairly well ($R^2 > 0.8$) should not be taken as a guarantee of success. That is because, as discussed in lecture 2, not all subpopulations of interest are equal: e.g., the model could be predicting a specific polymer class perfectly (polyesters) but fail badly in another one (polyethers). Thus, curating validation sets requires dividing the test set into slices that represent specific polymer families, with each validated independently.

2.2 Creating test cases

Another important principle discussed during the lectures concerns creating test cases. I don't have a background in software engineering, and so I often neglect writing functionality tests. Due to the rise of agentic systems for coding, the ease with which I can produce runnable code via prompting has become very tempting. The problem here is that code that runs successfully does not necessarily run as intended. For that reason, creating unit tests for almost all important functionalities has become a mandatory part of my development workflow. For example, encoding a polymer string into a graph and decoding it back should return the same structure.

3 Principles from the Guest Lectures

3.1 Hidden gaps become visible

During Per's lecture, the discussion went into a broad question: What will happen to software engineers? What resonated with me was that by automating a lot of routine work and writing code faster, many other hidden gaps become visible, such as evaluating undeclared model assumptions. A coding agent usually writes code with many undeclared assumptions unless instructed in detail. That process creates a responsibility for the reviewer to identify those assumptions and approve or reject them. In my research, I often find myself trying to preprocess chemical structures before they are ready for machine learning modelling. When an agent suggests the code, even if I specify the filtering steps, it can still introduce some assumptions I never mentioned, while the output still looks reasonable (e.g., valid polymers).

3.2 Levels of abstraction

With the current agentic tools, it is very tempting to go from an abstract idea to a working project. However, in research, this comes with some caveats. In my research, I could easily prompt an LLM saying: Build me a model that generates polymers suitable for batteries. The problem is that, to produce a reliable model, the prompt has to be further refined into: which property to optimize, what threshold choice is acceptable, how to trade off with other properties, etc. When I prompt at a higher level of abstraction, the agent does the refinement for me, and nothing points out that a decision was made. This connects back to the previous concept of hidden model assumptions: in both cases, the agent fills in decisions I never made, and the code runs either way.

4 Data Scientists, Software Engineers, and AI Engineers

The first two chapters of the book *Machine Learning in Production* [1] describe the differences between data scientists and software engineers in practice. Data scientists adopt a model-centric approach with feature engineering, model fitting, and model evaluation as the main concerns. As a consequence, a data scientist typically does not engage with system requirements, such as safety practices or reducing inference latency or operating cost. In contrast, software engineers focus more on the holistic design of a software system comprising multiple components and treat machine learning as one component of the overall system. The drawback here is that they typically lack the training to develop a robust machine learning pipeline that requires a data-centric approach along with domain knowledge.

I agree with the book's distinction because data scientists and software engineers are trained in distinct ways. As a data scientist, I was trained to focus on the details of machine learning, which include math, data cleaning and preprocessing, and optimization algorithms in depth. The holistic approach of designing system requirements was never ingrained in me throughout my studies.

In my research area, it is important that the pipeline I am building can be used later on by chemists easily. They have to be able to generate valid and optimized polymer structures that they can later evaluate with molecular simulations and experiments. This means that not only the model but rather the whole system must work as intended, because the chemists, representing the "customers" are directly using the outputs as part of their work. For example, parts of the non-ML pipeline such as preprocessing chemical structures and validity checks have to work correctly too and a fault in them would not appear as a drop in model accuracy. It is therefore essential to have a system-wide view, even though I am not building a business product.

Data science and software engineering, although branches of computer science, require different training from the ground up, and becoming knowledgeable in either takes years of practice. In my case, because I don't have an SE background, I tend to approach system design naively by ignoring infrastructure work. However, since I'm part of a small research group, I feel increasingly responsible for learning software engineering practices. This happens because I'm both a scientist and an engineer, though that may not be the case in industry, where roles are more clearly separated. To conclude, I believe that the two roles will remain distinct, but in some cases, someone has to be both.

In my opinion, AI engineering is software engineering applied to systems with ML components. As stated in the book, ML code is only a small fraction of the total software engineering code. An AI engineer is responsible for connecting the machine learning pipeline with the whole system while not going into the details of the model. The core of AI engineering is to design a system around the ML component so that when a model prediction is bad or unreliable, the system can respond to it. That is system requirements and architecture, which is, in principle, much closer to software engineering rather than data science.

Foundation models and modern agentic tools like coding assistants, RAG systems, and prompt pipelines can change the boundaries between software engineers and data scientists, but with a caveat. These tools reduce the barrier to producing work in the other discipline, but the user often cannot evaluate it. To provide a concrete example for my case, an agent may produce filtering code for polymers, and the output will look valid, but only domain knowledge can reveal what was discarded. So the boundary shifts from who can produce work to who can evaluate work. Evaluation is the skill that takes years to build, and agentic tools cannot substitute it.

5 Paper analysis

Paper 1: Themes of Building LLM-based Applications for Production: A Practitioner's View

While there is extensive academic research on incorporating Large Language Models (LLMs) in software engineering (SE) tasks, it isn't clear what practitioners should prioritize when it comes to deploying these systems in production. This paper revolves around exploring the themes of LLMs in production by analyzing videos of software practitioners using LLMs from 2022-2024 [2]. The themes include

Architecture and Design, Evaluation, Risks & Ethics, Monitoring & Observability, Costs, and Output Verification, among others. The authors identify RAG systems as the most frequently discussed topic in these themes, followed by evaluation and prompt engineering. An interesting point is that while practitioners heavily discuss the architecture design of LLMs, researchers often neglect to study that part. The authors argue that research should be directed towards the problems that software practitioners face, and that is, after all, where the difficulty in LLM systems resides.

The paper is not directly related to my research but provides a useful analogy. The authors note that public benchmark scores for LLMs are not particularly informative for LLMs in production systems. The practical alternative for that is to incorporate case-specific evaluation with humans in the loop. The same holds when I train machine learning models to predict polymer properties: the model can reach a very good metric on the held-out test set, but the real question is whether a trained chemist or I can judge a polymer as chemically sensible and synthesizable.

A larger AI-intensive software project that relates to my work could be an automatic polymer discovery platform used by an industrial battery lab. This can work as follows: a chemist describes what they need in detail (property target ranges, cost ceiling, etc.), the platform generates candidate polymer electrolytes, predicts their properties, and shortlists potential candidates. Then the chemist picks a few to simulate or synthesize, and the measured results are integrated back to retrain the property predictors. That workflow directly incorporates the paper's idea of human-in-the-loop evaluation into the generative pipeline. An output verification pipeline can also be integrated into the whole workflow by assessing deterministic validity and synthesizability through chemistry functions before the shortlist. My WASP research can include setting up the generative pipeline and the predictive models as well as the output verification pipeline and the user interface. That way, I can supply the ML and software components of the platform while the chemists supply the experimental validation of the loop.

At the current stage of my project, my pipeline is a collection of scripts to run my own experiments. Later on, I could use architectural design ideas to create an interface that connects all pieces of the pipeline together. In that case, a component change, e.g a different predictive model, can be swapped without rewriting the whole pipeline. Model swapping is important in my field mainly because polymer representations and models improve at a fast rate. With having a stable interface, one could rerun the same evaluation with a different predictor and compare directly.

Paper 2: Unified Software Engineering Agent as AI Software Engineer

This paper addresses the common problem of using multiple specialized agents for different software engineering tasks and attempts to create a unified agent to handle all tasks without losing efficacy [3]. The authors propose a universal agent that, instead of following a predefined workflow, decides its next action based on its current findings. Their experiments showed that the generalized agent performs as well as the specialized one on software benchmarks such as SWE-bench-verified (a benchmark where an agent fixes a GitHub issue and the tests must pass), suggesting that generality costs almost nothing. For the engineering of AI systems, one must decide whether an agent can run any command without following its trace or whether an agent should do a specialized task with fixed steps. This paper shows both: that you can have a generalized agent without hindering visibility of steps.

In their workflow, the authors examined 218 solutions that passed a specific test benchmark and found that approximately 10% were overfitting. The tests passed even though the patch was not correct. In

my polymer generation case, the generator produces polymers that are scored by a property predictor. However, in reinforcement learning, the process of reward hacking can occur, which is when the generator finds ways to maximize the reward without producing what I actually want. In my case, this is most likely to happen where the predictor is weaker, on polymer families with less training data. The generator is rewarded for high-scoring structures, so it has reason to drift into exactly those regions.

The polymer discovery platform I mentioned in Paper 1 can be extended using principles from this paper. I would use the unified agent for all the software engineering subtasks such as code review, unit testing, etc. Another functionality that I would add is to let it choose what to do next during polymer batch generation. For example, if a generated batch has poor validity or too many duplicates, the agent should autonomously decide to rerun the pipeline or make the necessary changes to ensure robustness. Moreover, since this will be an evolving platform, the agent can handle code maintenance, new tests, new requirements, and other things needed to support the healthy use of the platform. Finally, my research in WASP could supply the parts being orchestrated (e.g., preprocessing pipeline, predictors, generator).

At the current stage, the loop pipeline that I'm building is fixed, meaning that I generate candidates, score them, update the generative model's weights, and repeat. The paper's main argument is that the agent deciding the system's next step, rather than a predefined sequence, is superior. The long-term version of my project would replace the fixed loop with an autonomous one where the agent can intervene at any moment to perform any necessary changes.

6 Evidence, Originality, and GenAI-Use Transparency

Sources used

- Kaestner [1] book, online edition.
- Mailach et al. [2] peer-reviewed conference paper (CAIN 2025).
- Applis et al. [3] peer-reviewed conference paper (ICSE 2026).

The two conference papers are the ones I trust the most because both are peer-reviewed in highly impactful software conferences. The textbook is different from primary research, so I used it for framing and concepts rather than as evidence for specific claims.

Use of generative AI

I used Claude Sonnet across the whole essay for clarity, phrasing and brainstorming, as well as for helping me find the relevant papers in Section 5.

References

[1] Christian Kaestner. *Machine Learning in Production: From Models to Products*. MIT Press, Cambridge, MA, 2025. ISBN 9780262552462. URL `https://mlip-cmu.github.io/book/`.

[2] Alina Mailach, Sebastian Simon, Johannes Dorn, and Norbert Siegmund. Themes of building llm-based applications for production: A practitioner's view. In *2025 IEEE/ACM 4th International*

Conference on AI Engineering – Software Engineering for AI (CAIN), pages 18–30, 2025. doi: 10.1109/CAIN66642.2025.00011.

[3] Leonhard Applis, Yuntong Zhang, Shanchao Liang, Nan Jiang, Lin Tan, and Abhik Roychoudhury. Unified software engineering agent as ai software engineer. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, New York, NY, USA, apr 2026. ACM. doi: 10.1145/3744916.3773202.